REINS: Model-Free, Certifying Least-Cost Repair of LLM-Agent Tool-Call Trajectories

Anwar Debes
University of Agder
Draft v1.0, June 2026

Abstract

A runtime guardrail for an LLM agent faces a dilemma when it detects an unsafe action: a *detector* only reports the violation after the fact, and a *blocking* enforcer stops the action but, with it, the task. I present **REINS** (Runtime Enforcement of INterpretable Specifications), a certifying runtime *shield* that resolves this dilemma by computing, at each agent step, the *least-cost enactable edit* that keeps the agent's tool-call trajectory inside an interpretable bounded finite-trace-LTL policy's safe language (for a shieldable deployment; §4): it injects a missing prerequisite (an authorisation, an audit-log write), substitutes a safer tool variant, suppresses the call, or defers to a human. Of a conjunction of properties no prior system provides, two most sharply distinguish REINS from the 2025–2026 wave of LLM-agent enforcers. First, it is *model-free and CPU-only*: it edits the emitted tool-call stream *post hoc*, with no model in the repair loop, so it shields black-box and closed agents at a median of 225.70 μs per step. Second, it emits a *tamper-evident certificate of the enforcement action* (which clause fired, which edit was applied, that the edit was least-cost, and that the resulting trajectory is in the safe language) that an external auditor re-verifies *without the model*, catching not only an action let through unsafely (under-restriction) but also an action blocked that a cheaper safe edit would have allowed (over-restriction). I give the formal account: an *enactable edit automaton*, a soundness theorem and an identity-on-safe transparency theorem, and a static *shieldability* check; I am explicit that minimal-intervention enforcement, the soundness/transparency pairing, and the edit primitives are inherited from a deep line of work (edit automata [Ligatti et al., 2005], shield synthesis [Bloem et al., 2015, Alshiekh et al., 2018], optimal enforcement [Pandya and Wakankar, 2019, Gallay and Falcone, 2021]), and that my contribution is their specific *model-free, certifying* instantiation for agent tool calls. On a synthetic agent-task benchmark REINS dominates the safety/utility frontier: at attack success rate 0.000 (matching a blocking guardrail) it preserves Completion-under-Policy 1.000 versus 0.328 for blocking (a 3× utility gain at identical safety), and every committed edit is certifiably least-cost (1.000 of 8394 steps). I give an honest catalogue of failure modes: REINS is only as strong as its declared capabilities, its per-step optimum coincides with the whole-trajectory optimum on 0.975 of trajectories but not all, and the benchmark is synthetic. All numbers, figures and certificates regenerate on a single CPU.

1 Introduction

LLM agents now take consequential actions on a user's behalf (reading records, sending messages, moving money, running code), and the safety question has moved with them, from "is this text harmful?" to "is this *trajectory* of tool calls allowed?". The most damaging agent failures unfold over many steps: indirect prompt injection [Greshake et al., 2023], slow-roll data exfiltration, out-of-role tool use, missing compliance logging, and privilege escalation are all behaviours the OWASP Top 10 for Agentic Applications [OWASP Agentic Security Initiative, 2025] codifies, and the EU AI Act [European Parliament and Council, 2024], whose high-risk obligations apply from 2 August 2026, requires high-risk systems to log events over their lifetime (Article 12), remain robust (Article 15), and be overseeable by a human who can intervene (Article 14).

The enforcement dilemma. A guardrail that detects a temporal-policy violation must then *do* something. The two reflexes both fail the deployment. A pure *detector* (the dominant paradigm: Llama Guard, ShieldGemma, and the like) reports the violation but does not stop it. A *blocking* enforcer, which aborts the offending call or the session, is safe but pays a steep utility tax: it cannot tell a malicious transfer from a legitimate one that merely *omitted* a required authorisation step, so it blocks both. In my benchmark a blocking guardrail keeps only 0.328 of benign tasks completing under policy, because so many legitimate tasks trip a constraint that a small, safe correction would have satisfied.

The REINS approach. REINS is a runtime *shield* that, on a detected or imminent violation, computes the *least-cost enactable edit* that returns the trajectory to the policy's safe language, rather than blocking. Its edit alphabet, namely SUPPRESS, SUBSTITUTE (a declared safer variant), INSERT (a prerequisite the deployment can inject, e.g. an `authorize` or an `audit_log`), and DEFER (halt and ask a human), generalises the security automata of Schneider [Schneider, 2000] and the edit automata of Ligatti, Bauer and Walker [Ligatti et al., 2005] to the setting of an agent acting on *external tools*, where the enforcer does not fully control the action stream. I make the available edits a first-class, declarative *enactability relation*, choose among them by a cost model that prefers keeping the agent's intent, and *certify* the choice.

What is, and is not, new. Minimal-intervention enforcement is not new: shield synthesis [Bloem et al., 2015, Alshiekh et al., 2018], optimal run-time enforcement [Pandya and Wakankar, 2019], and decentralised LTL enforcement [Gallay and Falcone, 2021] all minimise interference and pair soundness with transparency, and least-cost correction to an automaton is a classical algorithm [Mohri, 2003]; certificates that an independent party re-checks are the certifying-algorithms discipline [McConnell et al., 2011]. I inherit all of this and say so plainly. REINS's contribution is a *conjunction* that, to my knowledge, no prior system provides: **(i)** a least-cost *enactable typed edit* over an agent's tool-call trajectory against a bounded-LTL safe language, computed **(ii)** *model-free and on CPU*, with no model re-invocation in the repair loop, so it works on closed/black-box agents, with **(iii)** proven soundness and identity-on-safe transparency, sealed into **(iv)** a tamper-evident *certificate of the enforcement action* re-verifiable without the model, and a **(v)** *measured safety/utility frontier* that dominates blocking. The two cleanest differentiators against the contemporary LLM-agent wave (§2) are (ii) and (iv): the nearest neighbour, the concurrent Agent-C [Kamath et al., 2025], repairs via in-decoder constrained generation (model-dependent) and emits no certificate; AgentSpec [Wang et al., 2026] can substitute/insert but only via hand-authored per-rule actions, with no least-cost objective, no soundness theorem, and no certificate.

Reproducibility. Every number, figure and certificate is produced by code in the accompanying repository on a single CPU; the experiment suite runs in minutes via `python experiments/run_all.py`.

2 Background and related work

Runtime enforcement: the lineage I inherit. Schneider's *security automata* [Schneier and Kelsey, 1998, Schneider, 2000] enforce safety by truncation (halt on the first bad step), and REINS's SUPPRESS/DEFER floor is exactly this. Ligatti, Bauer and Walker's *edit automata* [Ligatti et al., 2005] add insertion and suppression and introduce the soundness+transparency ("precise enforcement") pairing I use. Supervisory control [Ramadge and Wonham, 1987] frames enforcement as restricting a plant to a safe language by disabling controllable events. *Shield synthesis* [Bloem et al., 2015] and *safe RL via shielding* [Alshiekh et al., 2018, Könighofer et al., 2025] synthesise a reactive shield that overrides actions with *minimal interference*; Pandya and

Wakankar [2019] synthesise *H-optimal* shields minimising cumulative deviation, and Gallay and Falcone [2021] formalise soundness+transparency+*optimality* for LTL enforcement. Least-cost correction of a string to a (weighted) automaton is Mohri's [Mohri, 2003]. I claim novelty on *none* of these ideas; I instantiate them, model-free, for agent tool calls and certify the result.

Certifying computation. A *certifying algorithm* [McConnell et al., 2011] emits, with its output, a witness an independent checker verifies without trusting the producer. REINS's certificate is a certifying-enforcement witness: not a certificate of a *verdict* (as in certifying runtime monitors) but of an *action*, namely which edit ran, that it is least-cost, and that the result is in the safe language, re-checkable without the model. The tamper-evident substrate is standard secure logging [Schneier and Kelsey, 1998, Crosby and Wallach, 2009, Laurie et al., 2013, Krawczyk et al., 1997]; the contribution is what is bound into it.

The 2025–2026 LLM-agent enforcement wave. A burst of recent systems guards agents, and they split cleanly by *mechanism* (Table 1). *Detect / classify / verify, then block or alert:* GuardAgent [Xiang et al., 2024] and ShieldAgent [Chen et al., 2025] emit a safety label and reasons (the latter via probabilistic rule circuits), LlamaFirewall [Chennabasappa et al., 2025] runs content/code scanners, VeriGuard [Authors, 2025] verifies policy code offline then blocks/replans, and Pro2Guard [Wang et al., 2025] predicts an imminent violation from a learned Markov model and alerts/aborts; the related SAVeR [Yuan et al., 2026] repairs an agent's internal *belief* states (not the executable tool-call stream). None of these *repairs* the action stream with a least-cost edit or emits a per-intervention certificate. *Repair:* AgentSpec [Wang et al., 2026] substitutes/inserts via a hand-authored `invoke_action` per rule (no optimisation, no soundness theorem reported, no certificate); Agent-C [Kamath et al., 2025] (which appeared in December 2025, concurrent with and independent of this work, so I compare for completeness, not as a baseline) repairs, as described, by resampling a compliant alternative under grammar-/constrained decoding [Willard and Louf, 2023] (model-dependent), with no least-cost objective and no certificate. REINS occupies the model-free, least-cost, certifying corner of this design space. I use the agent-security benchmark AgentDojo [Debenedetti et al., 2024] and the Completion-under-Policy metric of ST-WebAgentBench [Levy et al., 2025] to ground (and scope) my evaluation.

Interpretable substrate. REINS's policy predicates are Boolean events over named tool capabilities: propositional, inspectable, and producible either by declarative adapters or by an interpretable Tsetlin-machine content guard [Granmo, 2018]. It shares this substrate with the author's companion systems THGTM [Debes, 2026a] (the learned primitive) and TRACE [Debes, 2026b]: where TRACE certifies a *detection verdict* (what a guard observed), REINS certifies an *enforcement action* (what the shield did), the enforcement layer TRACE left to future work. The paper is self-contained: nothing in the formal results or experiments depends on the companion systems.

3 Threat model

Setting. An agent executes a sequence of steps; at each step it issues a tool call. REINS sits inline as a *mediator* between the agent and the tools: it sees each call, may rewrite it, and the environment executes whatever REINS emits. It needs no access to the model's weights, logits, or chain of thought, only the tool-call wire format, which model-agnostic adapters normalise (OpenAI/DeepSeek `function`, Anthropic `tool_use`).

Policy. The deployment declares a policy P: a finite set of named constraints in a bounded fragment of finite-trace LTL (Always, Never, BoundedCount, Response, Precedence;

Table 1: LLM-agent guardrails by enforcement mechanism, summarising each cited paper's described design (not an audited re-implementation). "Model-free" = no model in the decision/repair loop (works on closed agents). "Least-cost" = chooses the minimal intervention automatically. "Sound." = the soundness guarantee reported ("conformance" / "PAC" = as the paper frames it; REINS's "yes" holds for a shieldable (P, C), Thm 1). "Cert." = emits a tamper-evident record re-verifiable without the model.

System	Mechanism	Model-free	Least-cost	Sound.	Cert.	CPU/μs
Llama Guard / ShieldGemma	classify	no	n/a	no	no	no
GuardAgent / ShieldAgent	verify+label	no	no	no	no	no
Pro2Guard	predict+alert/stop	no†	no	PAC	no	partial
Progent	privilege gate	yes	no	no	no	yes
AgentSpec	rule + `invoke_action`	yes	no	no‡	no	yes
Agent-C (concurrent)	in-decoder repair	**no**	no	conformance	no	no
REINS (this work)	**least-cost edit**	**yes**	**yes**	**yes***	**yes**	**yes**

†Pro2Guard learns its model offline but invokes no model at the decision point; I mark "no" as it is not a model-free repair. ‡AgentSpec is rule-driven and effective but reports no soundness theorem. *For a shieldable (P, C) (Thm 1); else the shield reports an enforcement failure or defers.

§4) over Boolean events, plus a *capability set* C stating which edits the deployment can enact (which prerequisites it can inject, which substitutes a tool accepts, whether a human escalation channel exists) and a cost model.

Adversary. A compromised or hijacked agent drives the trajectory toward harm: exfiltrating data over many small steps, acting out of role, omitting required logging, or performing privileged actions without authorisation, the OWASP agentic classes [OWASP Agentic Security Initiative, 2025]. I also consider an adversary who, after the fact, tampers with the audit log to hide what the shield did (or failed to do).

What REINS guarantees. (i) The executed trajectory satisfies P (Theorem 1), for a shieldable (P, C). (ii) On an already-compliant step the shield does nothing (Theorem 2). (iii) Each step's edit is least-cost among enactable safe edits (Proposition 1). (iv) Any post-hoc modification of the certificate is detected, and an auditor re-derives (i)–(iii) without the model.
What it does not. REINS enforces only the *declared* policy, over the *declared* events: a harmful behaviour nobody wrote a constraint for, or a substitute mis-declared as safe, is not prevented (it is, however, recorded; §5.6). It assumes the verification key is secret and the inline mediator itself is not compromised.

4 The REINS shield

4.1 Policy and monitors

A policy is a set of constraints over Boolean *events*, each compiled to an online, bounded-memory monitor of finite-trace LTL:

$$\textsc{Always}(p) \equiv \mathbf{G}\, p \qquad \textsc{Never}(e) \equiv \mathbf{G}\, \neg e$$
$$\textsc{BoundedCount}(e, k, W) \equiv \text{“} \leq k \ e \text{ per } W \text{ steps”} \qquad \textsc{Response}(t, r, d) \equiv \mathbf{G}(t \rightarrow \mathbf{F}_{\leq d}\, r)$$
$$\textsc{Precedence}(a, b) \equiv \neg b \ \mathbf{W}\ a$$

Each monitor latches a safety violation (a bad prefix cannot be undone) and uses memory bounded by its parameters, never by the trajectory length. Crucially for enforcement, every monitor exposes a side-effect-free one-step lookahead wouldViolate(events) and a cheap clone(), so the shield can ask "what would each candidate edit do?" without committing.

4.2 The enactable edit automaton

An *edit* rewrites the agent's candidate action a into an executed *sub-sequence* of actions:

$$\textsc{Allow} \to [a], \quad \textsc{Suppress} \to [\mathsf{noop}], \quad \textsc{Substitute}(a') \to [a'], \quad \textsc{Defer} \to [\,] \text{ (halt)},$$

and any edit may carry an *insert prefix* of prerequisite actions, e.g. inject `authorize` before a `transfer`, or `audit_log` before a `read`. The prefix is what preserves *utility*: the transfer still happens, after the authorisation the policy required. The set of *enactable* edits at a step is fixed by the capability set C: Allow and Suppress are always enactable (and Defer when an escalation channel exists); a $\textsc{Substitute}(a')$ is enactable iff a' is a declared safe variant of a; an $\textsc{Insert}(b)$ iff b raises an event in C's *insertable* set. Each edit has a cost; the default order encodes "keep the action if you can, halt only as a last resort":

$$\textsc{Allow} < \textsc{Insert} < \textsc{Substitute} < \textsc{Suppress} < \textsc{Defer}.$$

4.3 The repair planner

At each step the shield solves a tiny optimisation *exactly* by simulation (Algorithm in the repository): enumerate the enactable edits (a handful of base edits times the prerequisite prefixes, 9.44 candidates per step on the deployed policy), keep those whose emitted sub-sequence leaves *every* monitor safe (one constant-size $\mathsf{clone} + \mathsf{simulate}$ each), and return the least-cost survivor. There is no separate, untrusted "what would this edit do" oracle: candidate events are computed by the same adapter that maps the agent's own calls, so the planner's view of an edit is exactly what will execute.

4.4 Formal guarantees

Let the *invariant* I be "no monitor has ever reported a violation" (equivalently, the executed prefix is in the safe language of P). A policy P is *shieldable* under C if a safe enactable edit exists at every reachable step.

Theorem 1 (Soundness)**.** *If C provides escalation, then for* every *input action stream the REINS shield keeps I invariant. Without escalation the same holds provided P is* statically shieldable *under C: every $\textsc{Response}(t, r, d)$ has r insertable; every $\textsc{Always}(p)$ has p raised by the idle (noop) action or insertable; and C's insert budget $\geq$ the number of such obligations that can come due on one step. In either case the executed trajectory satisfies P.*

Proof sketch. Induction on steps. I holds on the empty trace. At a step, the feasible set $F = \{e : \mathsf{simulate}(e) \text{ preserves } I\}$ is non-empty: with escalation, Defer emits nothing and trivially preserves I. Without escalation, Suppress (emitting the idle noop) preserves Never, BoundedCount and Precedence unconditionally, and preserves $\textsc{Always}(p)$ exactly when the idle action raises p; an Insert supplies a p (or an expiring Response target r) the idle action does not, the inserts composing up to the budget. The shield commits $\arg\min_{e \in F} \mathsf{cost}(e)$, which preserves I. Hence I holds after every step. (The case where the idle action does not raise p, p is not insertable, and there is no escalation is exactly what static shieldability rules out; the shield then reports an enforcement failure rather than claiming success; see §5.6.) □

Theorem 2 (Transparency)**.** *If the agent's action a already keeps I invariant, the shield emits a unchanged (the identity).*

Proof. Allow (empty prefix) preserves I by assumption and has minimal cost 0; the planner's tie-break (least cost, then preference order, then fewest inserts) selects this empty-prefix Allow over any equal-cost edit. Hence the identity. (Under the default cost model Allow is the unique cost-0 edit; the tie-break covers cost models in which others are also free.) □

Proposition 1 (Minimal intervention)**.** *The committed edit is of least cost among enactable edits that keep I invariant at that step.*

This is by construction (the planner returns $\arg\min_{e \in F} \mathsf{cost}(e)$). It is a *per-step* (myopic) optimum; I do *not* claim global trajectory-optimality, and §5 measures how often the two coincide.

Proposition 2 (Constant cost)**.** *For a fixed policy, monitor state and per-step planning are constant in the trajectory length T; a session of any length runs in constant memory and $O(1)$ time per step.*

The candidate set and each monitor's state are bounded by the policy parameters, not by T (a ring buffer of size W for BoundedCount, $\leq d$ deadline counters for Response, a Boolean otherwise). One subtlety: an insert prefix of length L emits $L{+}1$ executed actions in one agent step, each a monitor transition, so a step costs $O(\mathsf{max_inserts})$ transitions (not strictly one) and deadlines/windows are measured in *emitted* actions, not agent calls; the planner and the committer use the same expansion, so soundness and transparency are unaffected.

Static shieldability. `is_shieldable`(P, C) decides in $O(|P|)$ whether soundness is guaranteed *without* halting (the non-degenerate case): every Response target must be insertable, every Always predicate must be idle-raised or insertable, and the insert budget must cover the constraints that can come due together (a conservative worst-case-co-occurrence count). Never, BoundedCount and Precedence are dischargeable by Suppress. The deployer is warned before deployment exactly where capabilities are inadequate; with an escalation channel soundness holds regardless (the shield halts). The check approximates the simultaneity bound conservatively rather than analysing reachable co-occurrence.

4.5 Certificate of the enforcement action

Each step is appended to a hash chain $h_i = \mathrm{SHA256}(h_{i-1} \,\|\, \mathrm{canonical}(\mathrm{payload}_i))$ with $\sigma_i = \mathrm{HMAC}_k(h_i)$, summarised by a Merkle root signed once. A payload records the candidate action, the chosen edit (its insert prefix, base, and substitute), the executed actions and their events, and the monitors that forced the edit. The certificate also embeds P, C, the cost model, and a model-free action→events map. An auditor holding the key re-verifies, *without the model*:

1. *Integrity*: recompute the chain and Merkle root; any modification, insertion, deletion or reordering breaks them (under SHA-256 collision resistance).
2. *Soundness*: replay the executed-event stream through the monitors; no monitor may ever violate.
3. *Minimal intervention*: *re-run the shield* from the embedded P, C and the recorded inputs, and confirm the recorded edit equals the recomputed least-cost safe edit at every step. This catches *under*-restriction (an unsafe action let through) *and over*-restriction (a blocked action a cheaper safe edit would have allowed).

This minimality witness certifies that the record matches the deterministic least-cost-edit *function* embedded in the certificate (P, C, the cost model, the action→events map): it detects record tampering and policy/capability misreporting. As with any certify-by-recomputation scheme, it is not an independent re-derivation of optimality against a *different* planner; a bug shared with the producer's planner would not be caught (the standard limitation of certifying algorithms [McConnell et al., 2011]). I provide *tamper-evidence* and symmetric authentication (HMAC), not non-repudiation; an Ed25519 signature over the Merkle root is the drop-in upgrade. Over-restriction is itself what optimal enforcement already minimises [Pandya and Wakankar, 2019, Gallay and Falcone, 2021]; what is, to my knowledge, new is an auditable, model-free

Table 2: E1: attack success rate (ASR, on attacks; lower better) and Completion-under-Policy (CuP, on benign tasks; higher better), 12 seeds. REINS matches the blocking ceiling on safety and triples utility.

Regime	ASR ↓	CuP ↑
no shield (unsafe baseline)	1.000	0.328
block (suppress/defer ceiling)	**0.000**	0.328 ± 0.078
REINS	**0.000**	$\mathbf{1.000 \pm 0.000}$

certificate from which an external party detects over-restriction *post hoc*, an accountability answer to the over-blocking that plagues agent guardrails.

5 Experiments

Protocol. All experiments run on a single CPU; latencies are `perf_counter` wall-clock. Unless noted I use 12 seeds and report mean ± std. The benchmark is a synthetic agent-task suite whose *shapes* follow the OWASP/Greshake taxonomy: benign tasks (some omitting a required prerequisite) and four attack families (exfiltration, destructive action, out-of-scope use, and missing/late prerequisites). I compare three regimes on identical tasks: `noshield` (observe only, execute everything), `block` (REINS with no utility-preserving capabilities, the blocking ceiling: suppress/defer only), and `reins` (the full shield). I score attack success rate (ASR, on attacks) and Completion-under-Policy (CuP, on benign tasks; a task counts only if its goal executed *and* no constraint was violated [Levy et al., 2025]). The benchmark is synthetic and not a live agent harness, a deliberate limitation (§5.6).

5.1 E1: the safety/utility frontier

This is the headline. Table 2 and Figure 1 show the three regimes. `noshield` executes every action (so the goal always runs) but is unsafe (ASR 1.000), so its *policy-compliant* completion is only CuP 0.328; `block` is safe (ASR 0.000) but keeps the same CuP 0.328 ± 0.078, since it blocks the many benign tasks that merely omit a prerequisite. `reins` is *both*: ASR 0.000 *and* CuP 1.000 ± 0.000, a 3× utility gain at identical (zero) attack success. A capability-richness sweep (Figure 1b) traces the frontier REINS moves along at fixed ASR = 0: suppress-only $0.328 \rightarrow$ +substitute $0.328 \rightarrow$ +insert-authorise $0.669 \rightarrow$ full 1.000. Figure 2 localises the block tax: it falls entirely on benign tasks that omit an authorisation or an audit log, which REINS repairs by injection and blocking destroys. *Caveat on the perfect score:* because the benign omissions and the deployment's insertable capabilities are drawn from the same taxonomy, CuP = 1.0 reflects a *best-case capability match*; on a live harness with un-anticipated omissions I expect CuP < 1, and the result should be read as "REINS recovers exactly the utility a correctly-capable deployment can", not as a universal guarantee (§5.6).

5.2 E2: soundness and shieldability

I fuzz 200 random adversarial trajectories per seed. On a shieldable deployment, 1.000 of shielded sessions are sound with 0 enforcement failures, Theorem 1 in practice. A deployment the static check flags as *non*-shieldable (a required audit not insertable, no escalation) does suffer failures (1777 over the suite, soundness 0.552); restoring the escalation channel returns soundness to 1.000 via deferral, the trivial Schneider guarantee (Figure 3, left). The static check predicts both, so the trade-off is visible before deployment.

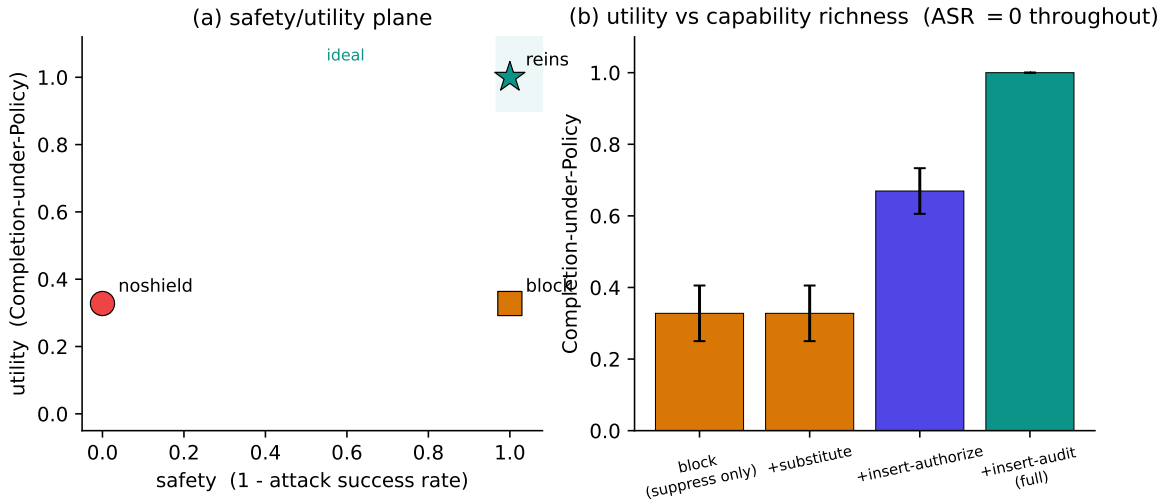


Figure 1: E1: (a) the safety/utility plane, where REINS sits in the ideal corner, matching blocking's safety with no-shield's utility; (b) CuP rises with capability richness, all at ASR = 0.

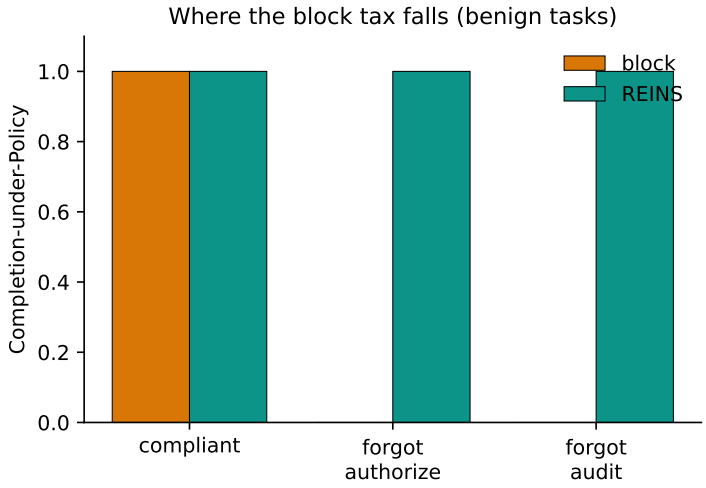


Figure 2: E1: the block tax falls on benign tasks that omit a prerequisite; REINS repairs them by injection.

5.3 E3: transparency and minimal intervention

On already-compliant streams the shield's intervention rate is 0.000: it is the identity (Theorem 2). Over 8394 steps of an adversarial fuzz, every committed edit equals the least-cost safe edit computed by a brute-force per-step oracle, a certified-minimal rate of 1.000 (Proposition 1, and what the certificate's minimality check re-derives). A branch-and-bound *global* optimiser on 480 short trajectories finds that the per-step optimum coincides with the whole-trajectory optimum on 0.975 of them: high, but *not* 1 (the largest myopic overspend was 15.0 cost units). I report this honestly rather than overclaim global optimality (Figure 3).

5.4 E4: the certificate

I seal sessions and apply six structural tamper classes plus a key-less forgery: all are detected (1.00; Figure 4a). In *validly-signed* logs I then plant non-minimal records: rewriting a substitute as a suppress (over-restriction) is caught at rate 1.00, and rewriting an intervention as an

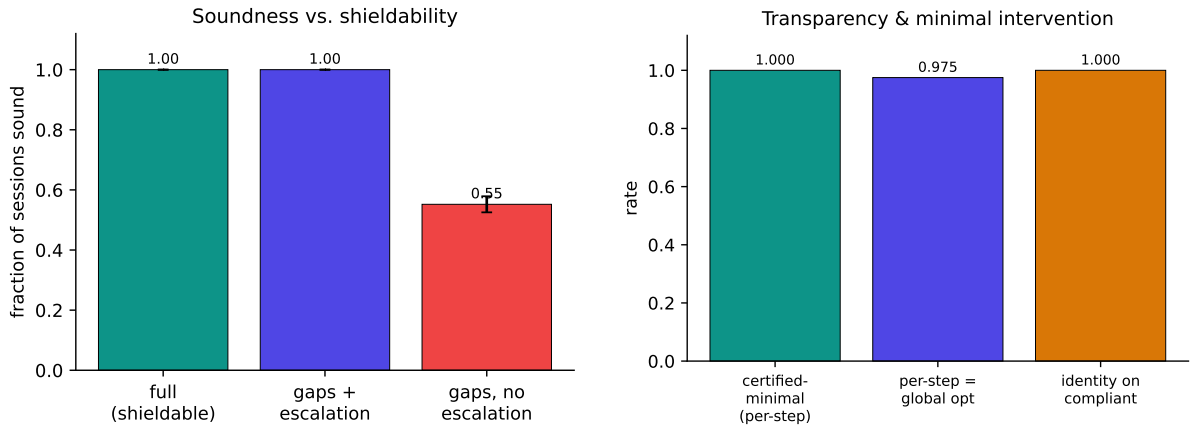


Figure 3: E2 (left): soundness tracks shieldability. E3 (right): the shield is identity on compliant streams, certifiably minimal per step, and matches the global optimum on 0.975 of trajectories.

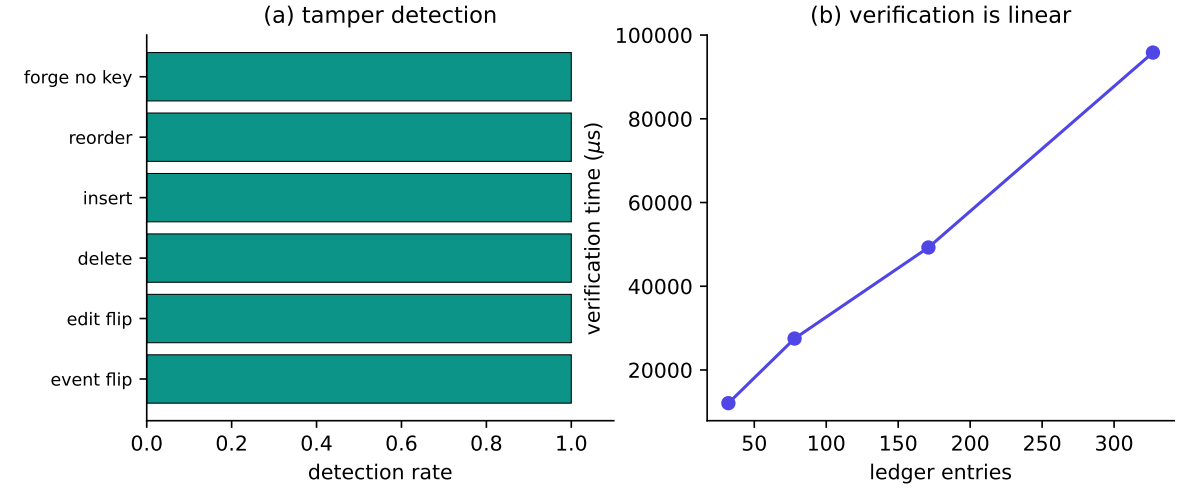


Figure 4: E4: (a) every tamper class is detected; (b) model-free verification is linear in ledger entries.

unguarded allow (under-restriction) is caught 12/12. The minimality re-verification flags both while integrity stays intact. Verification re-runs the entire shield model-free in 14278 μs for a 37-entry session and is linear in entries (Figure 4b).

5.5 E5: systems

A shield step costs 225.70 μs median (793.10 μs p99), enumerating 9.44 candidates; monitor state is 386 bytes and constant in trajectory length (Proposition 2); the ledger is $\sim$ 690 bytes per entry. Latency grows sub-linearly with the number of constraints (Figure 5) and stays well under a millisecond: REINS is as deployable inline as the monitors it enforces, on CPU.

5.6 E6: robustness and honest failure modes

Three probes, two adversarial to my own claims (Figure 6). *First*, an adaptive exfiltration attacker is *bounded*: REINS caps executed external sends at the rate budget (≤ 2) no matter how many are attempted, bounding the damage rather than being fooled. *Second*, REINS is only as strong as its declared capabilities: with the utility-preserving edits removed its benign CuP collapses to the blocking level (0.328 vs 1.000), and *without an escalation channel as well*

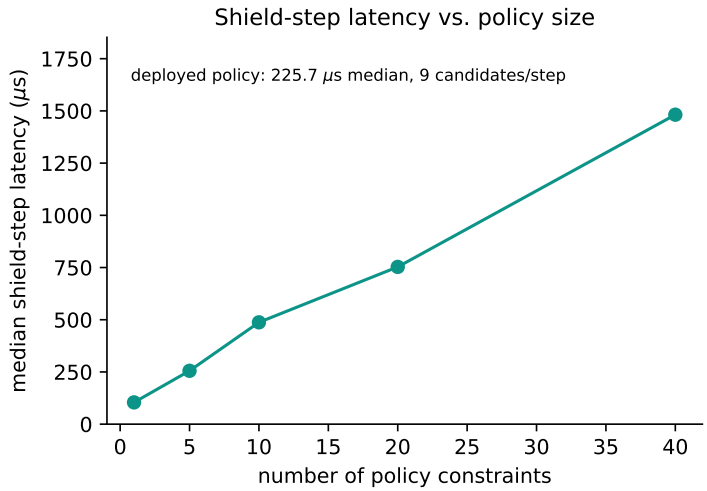


Figure 5: E5: shield-step latency vs. policy size; sub-millisecond on a CPU.

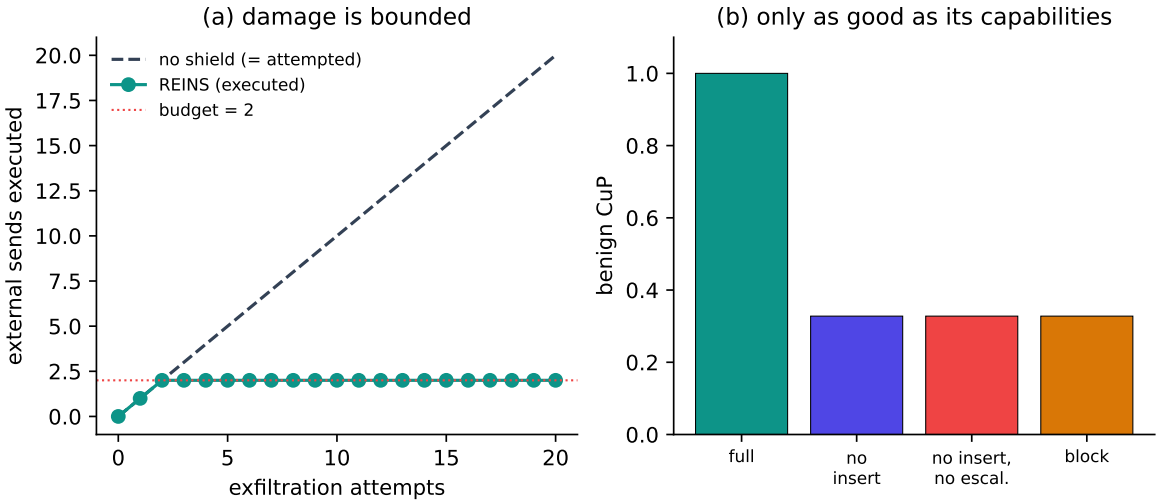


Figure 6: E6: (a) the rate monitor bounds exfiltration damage; (b) REINS is only as strong as its declared capabilities, so utility (and, without escalation, soundness) degrades as they are removed.

even soundness degrades (0.669), exactly the regime the static shieldability check warns against. *Third*, REINS guarantees safety with respect to the *declared* events, not un-modelled real-world effects: a mis-declared "safe" substitute is trusted, but it is recorded verbatim in the certificate (100% of substitutions are auditable), so the deployer's error is auditable even though it is not prevented. I also note the per-step/global gap of E3 and that the benchmark is synthetic.

5.7 E7: worked case studies

Four canonical situations, with the human-readable receipts REINS emits: a sensitive read missing its compliance log → INSERT(`audit_log`) at the deadline, read preserved; a transfer missing authorisation → INSERT(`authorize`), transfer proceeds; a destructive `delete_all` → SUBSTITUTE(`archive`); a slow-roll over-budget send → SUPPRESS. In each the blocking guardrail aborts the task; REINS keeps it moving while staying in the safe language, and the certificate records exactly what it did and why.

6 Discussion

REINS instantiates a *model-free*, *least-cost*, *certifying* repair shield, a combination the contemporary LLM-agent enforcers in Table 1 do not jointly provide. It does not need the model in the loop, so it shields closed and black-box agents on CPU; it preserves task utility where blocking destroys it; and it produces an auditable certificate that the agent was neither under- nor over-restricted, directly serviceable for EU AI Act record-keeping and human-oversight obligations. It is not a new enforcement theory: it stands on edit automata [Ligatti et al., 2005], shield synthesis [Bloem et al., 2015, Alshiekh et al., 2018], optimal enforcement [Pandya and Wakankar, 2019, Gallay and Falcone, 2021], and certifying algorithms [McConnell et al., 2011], and its value is their specific deployable, certifying conjunction for agents.

When to use REINS. Where an agent must keep working under a declared temporal policy and every intervention must be *explained and certified*: regulated deployments, on-device/edge agents, or a cheap auditable enforcement layer in front of a heavier neural guardrail.

7 Limitations and future work

- **Capability-bound.** REINS is only as strong as its declared capabilities and substitution table; inadequate capabilities collapse it toward blocking, and a mis-declared substitute is trusted (§5.6). The static check surfaces the former; the certificate makes the latter auditable.
- **Per-step, not global.** I prove and certify per-step minimality; the per-step optimum matched the whole-trajectory optimum on 0.975 of trajectories but not all. A bounded-lookahead planner is future work.
- **Declared-policy coverage.** REINS enforces only constraints someone wrote; mining bounded-LTL policies from agent logs [Lemieux et al., 2015] is the natural front-end.
- **Synthetic benchmark.** The trajectories follow published taxonomies but are not a live agent harness; an AgentDojo [Debenedetti et al., 2024] and ST-WebAgentBench [Levy et al., 2025] evaluation, and a head-to-head against AgentSpec and Agent-C, are future work.
- **Symmetric authentication only.** The reference ledger is HMAC-based; an Ed25519 signature over the Merkle root upgrades it to non-repudiation.

8 Ethics and broader impact

This work is defensive. The benchmark is synthetic and contains no personal data. I disclose REINS's failure modes (§5.6) precisely so deployers do not over-trust it: a shield that silently degrades to blocking, or trusts a wrong substitute, is dangerous if mistaken for complete. The interpretable policy and the certificate reveal the enforcement logic to an adversary; I judge the auditability benefit to outweigh this, consistent with open-source security practice.

9 Reproducibility

The repository contains the library, the unit-test suite, the seven experiments, the figure and number scripts, and this paper's source. `python -m pytest` runs the tests; `python experiments/run_all.py` regenerates every `results/*.json`; `python scripts/make_figures.py` and `paper_numbers.py` rebuild the figures and the macro file; the host, seeds and protocol are disclosed in §5.

10 Conclusion

A guardrail need not choose between seeing an unsafe action and stopping the task. REINS shows that a runtime shield can keep an LLM agent inside an interpretable temporal policy by the *least-cost enactable edit* (injecting a missing authorisation or audit, downgrading a dangerous tool, suppressing, or deferring), *model-free on CPU*, and seal each intervention into a certificate an auditor re-checks without the model, proving the agent was neither under- nor over-restricted. It dominates the safety/utility frontier over blocking at identical safety. Standing openly on the edit-automata and shielding lineage, its contribution is their deployable, certifying instantiation for agents, a concrete step toward guardrails that are not only effective but *accountable*.

References

Mohammed Alshiekh, Roderick Bloem, Rüdiger Ehlers, Bettina Könighofer, Scott Niekum, and Ufuk Topcu. Safe reinforcement learning via shielding. In *Proceedings of the 32nd AAAI Conference on Artificial Intelligence*, 2018. URL https://arxiv.org/abs/1708.08611. arXiv:1708.08611.

VeriGuard Authors. Veriguard: Enhancing llm agent safety via verified code generation. *arXiv preprint arXiv:2510.05156*, 2025. URL https://arxiv.org/abs/2510.05156.

Roderick Bloem, Bettina Könighofer, Robert Könighofer, and Chao Wang. Shield synthesis: Runtime enforcement for reactive systems. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, volume 9035 of *LNCS*, pages 533–548, 2015. URL https://arxiv.org/abs/1501.02573. arXiv:1501.02573.

Zhaorun Chen, Mintong Kang, and Bo Li. Shieldagent: Shielding agents via verifiable safety policy reasoning. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025. URL https://arxiv.org/abs/2503.22738. arXiv:2503.22738.

Sahana Chennabasappa, Cyrus Nikolaidis, Daniel Song, et al. Llamafirewall: An open source guardrail system for building secure ai agents. *arXiv preprint arXiv:2505.03574*, 2025. URL https://arxiv.org/abs/2505.03574.

Scott A. Crosby and Dan S. Wallach. Efficient data structures for tamper-evident logging. In *Proceedings of the 18th USENIX Security Symposium*, 2009.

Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024. arXiv:2406.13352.

Anwar Debes. Thgtm: A temporal hierarchical graph tsetlin machine with echo-trace automata for trajectory-level verification of agentic ai, 2026a. Reference implementation, v0.1.

Anwar Debes. Trace: A tsetlin runtime assurance and certification engine for interpretable, tamper-evident guardrails on agentic ai, 2026b. Reference implementation, v1.0.

European Parliament and Council. Regulation (eu) 2024/1689 of the european parliament and of the council laying down harmonised rules on artificial intelligence (artificial intelligence act). Official Journal of the European Union, L 2024/1689, 2024. URL https://eur-lex.europa.eu/eli/reg/2024/1689/oj.

Florian Gallay and Yliès Falcone. Decentralized ltl enforcement. In *Proceedings of GandALF 2021*, volume 346 of *EPTCS*, 2021. URL https://arxiv.org/abs/2107.06084. arXiv:2107.06084.

Ole-Christoffer Granmo. The tsetlin machine – a game theoretic bandit driven approach to optimal pattern recognition with propositional logic. *arXiv preprint arXiv:1804.01508*, 2018. URL https://arxiv.org/abs/1804.01508.

Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023. arXiv:2302.12173.

Adharsh Kamath, Sishen Zhang, Calvin Xu, Shubham Ugare, Gagandeep Singh, and Sasa Misailovic. Enforcing temporal constraints for llm agents. *arXiv preprint arXiv:2512.23738*, 2025. URL https://arxiv.org/abs/2512.23738.

Bettina Könighofer, Roderick Bloem, Nils Jansen, Sebastian Junges, and Stefan Pranger. Shields for safe reinforcement learning. *Communications of the ACM*, 68(11):80–90, 2025. URL https://cacm.acm.org/research/shields-for-safe-reinforcement-learning/.

Hugo Krawczyk, Mihir Bellare, and Ran Canetti. Hmac: Keyed-hashing for message authentication. RFC 2104, Internet Engineering Task Force, 1997. URL https://www.rfc-editor.org/rfc/rfc2104.

Ben Laurie, Adam Langley, and Emilia Kasper. Certificate transparency. RFC 6962, Internet Engineering Task Force, 2013. URL https://www.rfc-editor.org/rfc/rfc6962.

Caroline Lemieux, Dennis Park, and Ivan Beschastnikh. General ltl specification mining. In *Proceedings of the 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2015. URL https://www.cs.ubc.ca/~bestchai/papers/texada-ase15_final.pdf.

Ido Levy, Ben Wiesel, Sami Marreed, Alon Oved, Avi Yaeli, and Segev Shlomov. St-webagentbench: A benchmark for evaluating safety and trustworthiness in web agents. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025. URL https://arxiv.org/abs/2410.06703. arXiv:2410.06703.

Jay Ligatti, Lujo Bauer, and David Walker. Edit automata: Enforcement mechanisms for run-time security policies. *International Journal of Information Security*, 4(1–2):2–16, 2005. URL https://doi.org/10.1007/s10207-004-0046-8.

Ross M. McConnell, Kurt Mehlhorn, Stefan Näher, and Pascal Schweitzer. Certifying algorithms. *Computer Science Review*, 5(2):119–161, 2011. URL https://doi.org/10.1016/j.cosrev.2010.09.009.

Mehryar Mohri. Edit-distance of weighted automata: General definitions and algorithms. *International Journal of Foundations of Computer Science*, 14(6):957–982, 2003. URL https://doi.org/10.1142/S0129054103002114.

OWASP Agentic Security Initiative. Owasp top 10 for agentic applications. OWASP Foundation, 2025. URL https://genai.owasp.org/.

Paritosh K. Pandya and Amol Wakankar. Specification and optimal reactive synthesis of run-time enforcement shields. In *Proceedings of GandALF 2019*, volume 305 of *EPTCS*, 2019. URL https://arxiv.org/abs/1909.08541. arXiv:1909.08541.

Peter J. Ramadge and W. Murray Wonham. Supervisory control of a class of discrete event processes. *SIAM Journal on Control and Optimization*, 25(1):206–230, 1987. URL https://epubs.siam.org/doi/10.1137/0325013.

Fred B. Schneider. Enforceable security policies. *ACM Transactions on Information and System Security (TISSEC)*, 3(1):30–50, 2000. URL https://www.cs.cornell.edu/fbs/publications/EnfSecPols.pdf.

Bruce Schneier and John Kelsey. Cryptographic support for secure logs on untrusted machines. In *Proceedings of the 7th USENIX Security Symposium*, 1998.

Haoyu Wang, Christopher M. Poskitt, Jun Sun, and Jiali Wei. Pro2guard: Proactive runtime enforcement of llm agent safety via probabilistic model checking. *arXiv preprint arXiv:2508.00500*, 2025. URL https://arxiv.org/abs/2508.00500.

Haoyu Wang, Christopher M. Poskitt, and Jun Sun. Agentspec: Customizable runtime enforcement for safe and reliable llm agents. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2026. URL https://arxiv.org/abs/2503.18666. arXiv:2503.18666.

Brandon T. Willard and Rémi Louf. Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702*, 2023. URL https://arxiv.org/abs/2307.09702.

Zhen Xiang, Linzhi Zheng, Yanjie Li, Junyuan Hong, Qinbin Li, others, and Bo Li. Guardagent: Safeguard llm agents by a guard agent via knowledge-enabled reasoning. *arXiv preprint arXiv:2406.09187*, 2024. URL https://arxiv.org/abs/2406.09187.

Wenhao Yuan, Chenchen Lin, Jian Chen, Jinfeng Xu, Xuehe Wang, and Edith Cheuk Han Ngai. Verify before you commit: Towards faithful reasoning in llm agents via self-auditing. *arXiv preprint arXiv:2604.08401*, 2026. URL https://arxiv.org/abs/2604.08401.